



CHRONIQO

TEKNOLOGIANALYSE

SW7BAC-01 BACHELORPROJEKT

BILAG 4

Elev:

Mads Dittmann Villadsen

202209869 / AU715368

Vejleder:

Tommy Bjerre Nielsen

tbn@ece.au.dk

MAJ 2026

Indholdsfortegnelse

Tabelliste	iii
1. Introduktion til teknologianalyse	1
1.1. Formål og metode	2
1.2. Nøgleprincipper for teknologivalg	3
2. Backend-teknologi	4
2.1. Server-side-framework	5
2.1.1. Valg og begrundelse: Fullstack Type-safety	7
2.2. Databaseteknologi	8
2.2.1. Valg og begrundelse: relationel datamodel og dataintegritet	11
2.3. Autentificering og sikkerhed	12
2.3.1. Valg og begrundelse: sikkerhedsarkitektur og autentificeringsstrategi	14
2.4. Caching og session management	15
2.4.1. Valg og begrundelse: højtydende in-memory caching	17
2.5. Integration af AI og Large Language Models (LLM)	18
2.5.1. Valg og begrundelse: managed API med backend proxy-lag	20
2.6. Realtidskommunikation (chat)	21
2.6.1. Valg og begrundelse: short-polling som skalerbart kompromis	23
3. Frontend-teknologi	24
3.1. Frontend-framework	25
3.1.1. Valg og begrundelse: modent økosystem og fullstack-kapabilitet	28
3.2. Responsivt og adaptivt design	29
3.2.1. Valg og begrundelse: UI-framework og designimplementering	31
4. Udviklings- og deployment værktøjer	32
4.1. Versionsstyring (Git/GitLab)	33
4.1.1. Valg og begrundelse: repositorystruktur og workflow	35
4.2. Opgavestyring (Trello/Kanban)	36
4.2.1. Valg og begrundelse: opgavestyring og projektflyd	38
4.3. Hosting & deployment (cloud-platform)	39
4.3.1. Valg og begrundelse: cloud-infrastruktur og serverless hosting	41

4.4. UI/UX-designværktøjer	42
4.4.1. Valg og begrundelse: UI/UX-design og designsystem.....	44
4.5. Testværktøjer og kvalitetssikring	45
4.5.1. Valg og begrundelse: Jest som testframework	47
5. Samlet konklusion.....	48
5.1. Overblik over valgt teknologistak.....	49
5.2. Refleksion over teknologivalg (læring og udfordringer)	51
6. Referencer	52
7. Bilag.....	56

Tabelliste

Tabel 1: Sammenligning af server-side-frameworks	6
Tabel 2: Sammenligning af databaseteknologier	9
Tabel 3: Sammenligning af autentificeringsstrategier	13
Tabel 4: Sammenligning af caching-løsninger.....	16
Tabel 5: Sammenligning af LLM-teknologier	19
Tabel 6: Sammenligning af teknologier til realtidskommunikation.....	22
Tabel 7: Sammenligning af frontend-frameworks	26
Tabel 8: Sammenligning af CSS-tilgange	30
Tabel 9: Sammenligning af repositorystrukturer	34
Tabel 10: Sammenligning af opgavestyringsværktøjer.....	37
Tabel 11: Sammenligning af hosting-strategier	40
Tabel 12: Sammenligning af designværktøjer	43
Tabel 13: Sammenligning af testframeworks.....	46
Tabel 14: Samlet overblik over de valgte teknologier i Chroniqo	50

1. Introduktion til teknologianalyse

I dette afsnit præsenteres rammerne for de teknologiske valg, der danner fundamentet for udviklingen af systemet. Formålet med teknologianalysen er at sikre, at systemarkitekturen baseres på velbegrundede beslutninger, der understøtter projektets kravspecifikation og de identificerede risici.

Analysen fokuserer på at identificere, vurdere og udvælge de mest hensigtsmæssige teknologier til både backend, frontend og infrastruktur, så der skabes en robust, skalerbar og vedligeholdelsesvenlig løsning.

1.1. Formål og metode

Formålet med denne analyse er at bevæge sig væk fra subjektive præferencer (hvad jeg kender) og i stedet træffe systematisk begrundede valg baseret på projektets specifikke behov. Metoden for udvælgelsen følger en struktureret proces for hver hovedkomponent i systemet:

- **Identifikation af kandidater:** Der opstilles relevante teknologiske alternativer (f.eks. forskellige frameworks eller databasetyper), der er udbredte i industrien og potentielt kan løse opgaven.
- **Vurdering:** Kandidaterne evalueres ud fra en række nøglekriterier (se afsnit 1.2), hvor deres styrker og svagheder holdes op imod systemets specifikke krav, såsom behovet for rapid prototyping og høj datasikkerhed.
- **Valg og begrundelse:** På baggrund af vurderingen træffes det endelige teknologivalg, og beslutningen begrundes med fokus på, hvordan teknologien bedst understøtter projektets mål.

1.2. Nøgleprincipper for teknologivalg

For at sikre en ensartet evaluering på tværs af alle teknologikategorier, er følgende nøgleprincipper defineret som styrende for udvælgelsen:

- **Developer Experience (DX) og Effektivitet:** Da projektet udføres af en enkeltmandsudvikler inden for en begrænset tidsramme, vægtes teknologier, der tilbyder høj produktivitet, stærk type-sikkerhed (f.eks. via TypeScript (1)) og et modent økosystem (frameworks med omfattende standardbiblioteker og stærk community-support). Dette minimerer unødigt kompleksitet og risikoen for tidskrævende konfigurationsproblemer (jf. R11 i risikoanalysen (2)).
- **Skalerbarhed og Performance:** Selvom systemet starter som en MVP, skal teknologistakken kunne håndtere en potentiel vækst i brugerbasen uden behov for fundamental omskrivning. Løsningen skal kunne understøtte asynkron kommunikation og hurtig dataudveksling.
- **Datasikkerhed og Integritet:** Da systemet håndterer potentielt følsomme data om brugernes helbred og trivsel, prioriteres teknologier, der understøtter stærk datakonsistens (ACID-principper (3)) og etablerede sikkerhedsstandarder.
- **Vedligeholdelse og Dokumentation:** Der foretrækkes teknologier med omfattende dokumentation og store fællesskaber, hvilket sikrer, at løsningen er fremtidssikret og lettere at overdrage eller vedligeholde.

2. Backend-teknologi

I dette afsnit analyseres og udvælges systemets backend-teknologier. Dette inkluderer valg af runtime-miljø/framework, database og autentificeringsmekanismer.

2.1. Server-side-framework

Valget af backend-framework er kritisk for systemets ydeevne og udviklingshastighed. Da systemet skal håndtere både API-kald, databaseinteraktioner og potentielt realtids-kommunikation, kræves en platform, der er robust og fleksibel.

Følgende Tabel 1 sammenligner tre udbredte backend-teknologier: Next.js (App Router), Node.js (Express) (Microservice) og Python (Django):

Tabel 1: Sammenligning af server-side frameworks baseret på fordele og ulemper i forhold til projektets krav.

Server-side framework			
Teknologi	Fordele	Ulemper	Valgt?
Next.js (App Router) (Fullstack) (4)	Simplicitet: Samler frontend og backend i én samlet kodebase. Ingen CORS-problemer. Type-safety: 100 % end-to-end typesikkerhed med Prisma (5) uden behov for eksterne pakker.	Vendor Lock-in: Tæt bundet til Vercels økosystem. State: Serverless natur gør stateful forbindelser (som WebSockets) sværere at implementere.	☒
Node.js (Express) (Microservice) (6, 7)	Fleksibilitet: Høj grad af kontrol over arkitekturen og middleware. Agnostisk: Kan hostes overalt uafhængigt af frontenden.	Overhead: Kræver vedligeholdelse af to separate repositories, to CI/CD-pipelines og dobbelt konfiguration. R1 (2) Risiko: Markant øget tidsforbrug.	☐
Python (Django / FastAPI) (8)	Udviklingshastighed: Hurtig opsætning med indbyggede moduler ("batteries included"). Økosystem: Industristandard for integration af evt. fremtidig AI/ML.	Context-switching: Kræver skift mellem Python (backend) og Typescript (1) (frontend). Arkitektur: Django kan føles rigtigt, hvis man ønsker microservices eller fuld kontrol over routing.	☐

Ud fra ovenstående tabelsammenligninger, er valget faldet på Next.js (App Router) som det primære framework til både frontend og backend. Ved at samle arkitekturen i en monolit frem for en opdelt Express-struktur sikres en højere projektfremdrift, da kompleksiteten i konfiguration og data-delning minimeres.

2.1.1. Valg og begrundelse: Fullstack Type-safety

Valget af en samlet Next.js-arkitektur bunder i en strategi om *Architectural Simplification*, der sigter mod at eliminere den kognitive og arkitektoniske kompleksitet, som normalt er forbundet med at vedligeholde to separate kodebaser. Ved at samle frontend og backend i én monolit fjernes behovet for at håndtere komplekse CORS-konfigurationer og manuelt synkronisere datatyper over netværket. I stedet udnyttes Next.js Route Handlers (`app/api/.../route.ts`) (9), hvilket muliggør en direkte og typesikker integration med Prisma ORM (Object-Relational Mapper) (5) uden et mellem-liggende API-lag.

Hvor en traditionel adskillelse mellem en Express.js-backend og en React-frontend (Poly-repo) kræver omfattende standardiseret opsætning til kontraktstyring og autentificerings-handshakes, tillader Next.js-monolitten en tættere kobling af systemets komponenter.

Selvom en distribueret arkitektur tilbyder fordele i enterprise-skala, vurderes det, at denne model maksimerer Developer Experience (DX) og projektfremdriften for en enkeltmandsudvikler. Ved at reducere antallet af variable faktorer i infrastrukturen minimeres den tekniske kompleksitet, hvilket aktivt beskytter projektets eksekveringsevne mod uforudsete forhindringer, uden at gå på kompromis med systemets skalerbarhed som MVP.

2.2. Databaseteknologi

Databasen er rygraden i systemet, da den skal opbevare og sikre integriteten af brugernes data, opslag og relationer. Valget står mellem traditionelle relationelle databaser (SQL), dokument-orienterede databaser (NoSQL) og cloud-baserede løsninger.

Følgende Tabel 2 sammenligner fire udbredte databaseteknologier: PostgreSQL, MySQL, MongoDB og Google Firebase (Firestore):

Tabel 2: Sammenligning af databaseteknologier baseret på fordele og ulemper i forhold til projektets krav.

Databaseteknologi			
Teknologi	Fordele	Ulemper	Valgt?
PostgreSQL (SQL) (10)	Avanceret funktionalitet: Stærk support for komplekse relationer, JSONB (hybrid SQL/NoSQL) og udvidelser. Dataintegritet: Streng ACID-compliance (3). Økosystem: Fungerer fremragende med moderne ORM'er som Prisma (5).	Kompleksitet: Kan være tunge at konfigurere og tune end MySQL. Skalering: Horisontal skalering kræver mere opsætning end NoSQL.	☒
MySQL (11)	Udbredelse: Verdens mest anvendte open-source database, meget nem at finde hosting til (12). Hastighed: Ofte hurtigere til simple læse-operationer. Simplicitet: Lettere læringskurve end Postgres.	Funktionalitet: Mindre robust support for komplekse datatyper og JSON sammenlignet med Postgres. Licens: Ejers af Oracle (MariaDB er dog open source-fork).	☐
MongoDB (13)	Fleksibilitet: Skemaløs (nemt at ændre datastruktur "on the fly"). Skalering: Bygget til nem horisontal skalering (sharding). Hastighed: Hurtig til simple læsninger/skrivninger af enkelte dokumenter.	Ingen JOINS: Relationer skal håndteres manuelt i applikationskoden, hvilket øger kompleksiteten. Datakonsistens: Svagere garantier for referentiel integritet (risiko for orphaned data).	☐

Google Firebase (Firestore) (14)	Real-time: Indbygget synkronisering til klienter (godt til realtidskommunikation). Hastighed: Ingen server-setup (Serverless/BaaS). Skalering: Håndteres automatisk af Google.	Vendor Lock-in: Meget svært at migrere væk fra Google senere. Begrænsede forespørgsler: Relationelle queries (JOIN-lignende) understøttes ikke direkte; komplekse filtreringer kræver ofte data-duplicering eller ekstra applikationslogik. Pris: Kan blive dyrt ved mange læse/skrive-operationer.	
Supabase (Postgres BaaS) (15)	Hastighed: Giver en fuld Postgres DB + Auth + API med det samme. Features: Realtime subscriptions ud af boksen. Open Source: Ingen vendor lock-in på samme måde som Firebase.	Abstraktion: Skjuler noget af infrastrukturen, hvilket mindsker læringsværdien i et ingeniørprojekt. Begrænsninger: Kan være sværere at tilpasse med custom backend-logik end en ren Postgres container.	

Ud fra ovenstående tabelsammenligninger er valget faldet på PostgreSQL som databaseteknologi.

2.2.1. Valg og begrundelse: relationel datamodel og dataintegritet

Et socialt netværk er fundamentalt set bygget op omkring relationer:

- En **Bruger** har mange **Opslag**.
- Et **Opslag** har mange **Kommentarer**.
- En **Kommentar** tilhører en **Bruger** og et **Opslag**.

I en relationel database som PostgreSQL håndteres disse forbindelser naturligt via Foreign Keys og JOINS. Hvis en bruger slettes, kan databasen automatisk sørge for at rydde op i (eller anonymisere) deres opslag gennem regler som `ON DELETE CASCADE`, hvilket sikrer høj dataintegritet.

I en NoSQL-løsning som MongoDB ville man ofte være tvunget til at duplikere data (f.eks. gemme brugernavn og avatar direkte på hver kommentar) for at undgå dyre opslag. Hvis brugeren så ændrer navn eller billede, skal man manuelt opdatere tusindvis af dokumenter, hvilket er fejludsat og komplekst at vedligeholde.

Denne arkitektur gør det muligt at designe en datamodel, der er bygget op omkring kernerelationer som Bruger, Opslag og Kommentar. Ved at lægge integritetsreglerne direkte i databasen fremfor kun i koden, skabes et langt mere robust fundament. Det gør det betydeligt lettere at udvide med funktioner som avanceret søgning og moderation senere hen.

Desuden fungerer PostgreSQL fremragende sammen med Prisma ORM (5). Prisma kan introspecte SQL-skemaet og generere TypeScript-typer automatisk, hvilket giver den ønskede typesikkerhed helt ned i databaselaget.

2.3. Autentificering og sikkerhed

Sikker håndtering af brugeridentiteter er en fundamental del af systemet. Valget af autentificeringsstrategi skal balancere brugervenlighed med høj sikkerhed mod angreb som CSRF (Cross-Site Request Forgery) og XSS (Cross-Site Scripting).

Følgende Tabel 3 sammenligner tre tilgange til autentificering: Session-baseret (Cookies), Token-baseret (JWT) og Managed Auth (e.g. Clerk):

Tabel 3: Sammenligning af autentificeringsstrategier baseret på fordele og ulemper i forhold til projektets krav.

Autentificering og sikkerhed			
Teknologi	Fordele	Ulemper	Valgt?
Auth.js (NextAuth v5) (16)	Sikkerhed: Håndterer JWT, HttpOnly-cookies og CSRF-beskyttelse "out of the box". Integration: Skræddersyet til Next.js. Nem opsætning af Google OAuth.	Black-box: Abstraherer meget logik væk, hvilket begrænser muligheden for detaljeret konfiguration sammenlignet med en egenudviklet løsning.	☒
Egenudviklet OAuth / PKCE	Fuld kontrol: Giver dyb indsigt i og kontrol over det kryptografiske flow. Fleksibilitet: Helt uafhængig af specifikke biblioteker.	Risiko: "Never roll your own crypto" (17). Enormt tidskrævende at bygge sikkert fra bunden. Stor risiko for sårbarheder.	☐
Managed Auth (e.g. Clerk (18))	Sikkerhed: Håndteres af eksperter. Features: MFA, Social Login m.m. er indbygget. Hastighed: Meget hurtig implementering.	Pris: Kan blive dyrt ved mange brugere. Afhængighed: Skaber en kritisk afhængighed af en ekstern leverandør. Kontrol: Mindre kontrol over brugerdata og flow.	☐

Ud fra ovenstående tabelsammenligninger, er valget faldet på Auth.js (NextAuth v5) til håndtering af systemets autentificering af brugere.

2.3.1. Valg og begrundelse: sikkerhedsarkitektur og autentificeringsstrategi

For at imødekomme systemets sikkerhedskrav og minimere risikoen for sårbarheder (jf. R13: Sikkerhedshuller (2)) implementeres autentificeringen via Auth.js (NextAuth v5). Inden for softwarearkitektur følges princippet om at anvende anerkendte sikkerhedsstandarder frem for egenudviklet kryptografi og autentificeringslogik. Implementeringen af et komplet OAuth 2.0 PKCE-flow (19, 20) fra bunden er en kompleks og ressourcekrævende proces, hvor risikoen for sikkerhedsbrister er væsentligt forhøjet sammenlignet med anvendelsen af gennemtestede biblioteker.

Ved at benytte Auth.js udliciteres ansvaret for session-håndtering, generering af `HttpOnly`-cookies og CSRF-beskyttelse til en standardiseret løsning. Dette valg muliggør både lokal validering via Credentials provider og integration med Google OAuth, samtidig med at den konfigurationsmæssige byrde understøtter risikohåndteringen for R1: Helbredsudfordringer (2).

Arkitekturen sikrer således en robust sikkerhedsmodel, uden at projektets fremdrift belastes af unødigt teknisk kompleksitet i infrastrukturen.

2.4. Caching og session management

For at sikre systemets responsivitet og beskytte mod overbelastning er det nødvendigt med en hurtig mellemlagring af data.

Følgende Tabel 4 sammenligner tre tilgange til hurtig mellemlagring af data: In-Memory (RAM), Redis og Database-opslag:

Tabel 4: Sammenligning af caching-løsninger baseret på fordele og ulemper i forhold til projektets krav.

Caching og session management			
Teknologi	Fordele	Ulemper	Valgt?
Redis (Key-Value Store) (21)	Hastighed: Ekstremt hurtig (sub-millisekund) læsning/skrivning. Persistens: Data kan overleve genstart af applikationen. Funktionalitet: Indbygget TTL (Time-To-Live) til automatisk sletning af data (perfekt til rate limiting).	Infrastruktur: Kræver en ekstra service/container i drift. Kompleksitet: En smule mere opsætning end simpel in-memory.	☒
In-Memory (Node.js variabler)	Simplicitet: Ingen ekstra infrastruktur kræves. Hastighed: Hurtigste løsning (direkte RAM-adgang).	Skalering: Data deles ikke mellem flere server-instanser. Flygtighed: Data mistes ved genstart af serveren.	☐
Database (SQL)	Simplicitet: Ingen ekstra infrastruktur (bruger eksisterende DB). Persistens: Data er gemt sikkert på disk.	Performance: Langsommere end RAM. Dårligt egnet til højfrekvente tællere (f.eks. rate limiting), da det belaster disken unødigt.	☐

Ud fra ovenstående tabelsammenligninger, er valget faldet på Redis som caching- og session-store.

2.4.1. Valg og begrundelse: højtydende in-memory caching

Valget af Redis (21) er primært drevet af to kritiske sikkerhedsbehov: Rate Limiting og beskyttelse mod Brute-Force-angreb.

For at beskytte Next.js API'et mod misbrug, er det nødvendigt at kunne tælle og begrænse antallet af forespørgsler pr. IP-adresse i realtid. Redis' in-memory arkitektur (21) gør det muligt at håndtere trafik uden mærkbar forsinkelse (latency), og uden at overbelaste PostgreSQL-databasen med læse/skrive-operationer for hver eneste request.

Selvom selve brugeradgangen og sessionerne nu håndteres sikkert og stateless via Auth.js (NextAuth) (16), er selve applikationslaget stadig sårbart overfor overbelastning (DDoS eller scraping). Ved at integrere Redis med indbygget Time-To-Live (TTL) (21) sikres det, at blokeringer af IP-adresser automatisk ryddes op efter få minutter. Dette holder backenden robust og muliggør horisontal skalering på Vercels Edge-netværk (22), da Rate Limit-tælleren (*state*) er løftet ud af den enkelte serverinstans.

2.5. Integration af AI og Large Language Models (LLM)

Selvom den empatiske AI-støttebot er prioriteret som et Could-krav (jf. US3.19 (23)), er det nødvendigt at fastlægge den teknologiske retning for at sikre, at arkitekturen kan understøtte funktionen. For et MVP-projekt er de primære valgte kriterier, at løsningen er gratis (eller har en meget generøs gratis udviklertier), at den nemt kan integreres i det eksisterende miljø, og at den overholder projektets privatlivskrav.

Følgende Tabel 5 sammenligner tre tilgange til LLM-integration: OpenAI API, Google Gemini API og Self-hosted Open Source (Llama 3 via Ollama):

Tabel 5: Sammenligning af LLM-teknologier baseret på fordele og ulemper i forhold til projektets krav om nul-omkostninger og serverless infrastruktur.

Integration af AI og Large Language Models (LLM)			
Teknologi	Fordele	Ulemper	Valgt?
Google Gemini API (Gemini 2.5 Flash Lite) (24)	Pris: Tilbyder en yderst generøs gratis udviklertier (perfekt til MVP). Hastighed: Meget lav latency, hvilket understøtter et responsivt UI.	Privacy: I den gratis tier kan data potentielt bruges til modeltræning af Google, hvilket kræver stram datarensning.	☒
OpenAI API (GPT-4o-mini) (25)	Kvalitet: Industri-standard med fremragende Developer Experience (DX). Privacy: API-data bruges (som standard) ikke til at træne deres modeller.	Pris: Har ingen reel gratis tier (kræver betalingskort). Selvom det er billigt, bryder det med kravet om nul-omkostninger.	☐
Self-hosted Open Source (f.eks. Llama 3 via Ollama) (26)	Data-suverænitet: 100 % privacy, da data aldrig forlader egen server. Licens: Gratis software uden vendor lock-in.	Infrastruktur: Kræver dedikeret GPU-hardware for at køre acceptabelt. Dette strider direkte imod den valgte serverless cloud-strategi (Vercel) og introducerer høje driftsomkostninger.	☐

Ud fra ovenstående tabelsammenligninger, er valget faldet på en Managed API-løsning (Google Gemini API) til systemets AI-funktioner.

2.5.1. Valg og begrundelse: managed API med backend proxy-lag

Valget er begrundet i et strengt krav om nul driftsomkostninger til projektets MVP, hvilket udelukker betalte API'er som OpenAI. Samtidig fravælges self-hosted open-source modeller (f.eks. Ollama), da kravet om dedikeret GPU-hardware strider direkte imod systemets valgte serverless arkitektur (Vercel) og ressourcekapaciteten i et enkeltmandsprojekt.

For at forebygge privacy-bekymringer ved brug af et gratis eksternt API, implementeres integrationen udelukkende server-side via Next.js Route Handlers (9). For MVP'en håndteres PII-filtrering (Personhenførbare Data) pragmatisk: PII-beskyttelsen implementeres udelukkende i Service-laget via en Regex-baseret pre-filtrering, der identificerer og erstatter e-mailadresser, telefonnumre, CPR-numre og brugerens eget navn med neutrale placeholders (f.eks. [NAME REMOVED]), inden prompten videresendes til Gemini API'et. System Prompten instruerer Niqo, platformens støttebot, i aldrig at afsløre over for brugeren, at denne filtrering finder sted, men fungerer ikke i sig selv som en teknisk PII-barriere - det er Regex-filteret, der bærer den ansvaret.

For at understøtte en kontinuerlig og meningsfuld samtale (jf. US3.19 (23)) persisteres chathistorikken i PostgreSQL via to dedikerede modeller, NiqoChat og NiqoMessage. Systemet begrænser lagringen til brugerens seneste samtale; påbegyndes en ny chat, slettes den foregående automatisk, og brugeren kan desuden manuelt slette samtalen. ON DELETE CASCADE på begge modeller sikrer automatisk oprydning af samtaledata, hvis en brugerkonto slettes, i overensstemmelse med GDPR.

2.6. Realtidskommunikation (chat)

Systemet kræver understøttelse af 1:1-, gruppe- og fællesskabschats (jf. US3.1 (23)). Da arkitekturen baseres på en serverless Next.js platform (hostet på Vercel), er traditionelle stateful WebSockets problematiske at implementere. Årsagen til dette er, at WebSockets kræver en vedvarende (persistent), åben TCP-forbindelse mellem klient og server. Serverless-funktioner er derimod designet til at være kortlivede; de starter op (cold start) for at håndtere et specifikt request og lukker ned umiddelbart efter for at spare ressourcer. Dette gør det umuligt at opretholde den lange forbindelse, som WebSockets kræver, uden brug af en separat, dedikeret realtids-infrastruktur.

Følgende Tabel 6 sammenligner tre tilgange til realtidskommunikation: Short-Polling (SWR), WebSockets (Socket.io/WS) og Managed Realtime (Pusher/Supabase):

Tabel 6: Sammenligning af teknologier til realtidskommunikation baseret på projektets rammevilkår.

Realtidskommunikation (chat)			
Teknologi	Fordele	Ulemper	Valgt?
Short-Polling (SWR) (27, 28)	Simplicitet: Kræver ingen ekstra infrastruktur eller stateful server. Pålidelighed: Fungerer fejlfrit i serverless miljøer og genoptager automatisk forbindelsen.	Skalerbarhed: Øget pres på database og netværk ved mange samtidige brugere. Latency: En lille forsinkelse svarende til fetch-intervallet.	☒
WebSockets (Socket.io / WS) (29)	Realtid: Ægte tovejskommunikation med minimal forsinkelse. Effektivitet: Lavt overhead efter den initiale forbindelse er etableret.	Infrastruktur: Kræver en dedikeret Node.js-server (ikke serverless). Kompleksitet: Svært at vedligeholde og skalere i en monolitisk Next.js-struktur.	☐
Managed Realtime (Pusher / Supabase) (15, 30)	Performance: Håndterer al trafik eksternt og aflaster systemets egen database. Features: Indbygget understøttelse af presence (hvem er online).	Pris & Lock-in: Introducerer driftsomkostninger og en afhængighed af en tredjepartsplatform.	☐

Ud fra ovenstående tabelsammenligninger er valget faldet på en Short-Polling løsning implementeret via SWR, der fetcher nye beskeder med få sekunders interval, til systemets realtidskommunikation mellem brugere.

2.6.1. Valg og begrundelse: short-polling som skalerbart kompromis

For MVP'en træffes der et bevidst arkitektonisk kompromis ved at implementere realtidskommunikationen ved hjælp af en Short-Polling løsning. Selvom polling ikke er optimalt skalerbart til titusindvis af samtidige brugere, er det fuldt ud tilstrækkeligt til at validere konceptet for op til 50 samtidige brugere (jf. kvalitetskrav PR2: Forringelse af svartider i chat-funktionen (23)). Hvis systemet skaleres i fremtiden, vil arkitekturen skulle udvides med en Managed Realtime Service (såsom Pusher eller Supabase Realtime) for at aflaste databasen og minimere netværkstrafik.

3. Frontend-teknologi

I dette afsnit analyseres og udvælges de teknologier, der skal udgøre klientsiden af systemet. Fokus er på at vælge værktøjer, der sikrer en moderne, responsiv brugeroplevelse og understøtter en effektiv udviklingsproces.

3.1. Frontend-framework

Valget af frontend-framework er afgørende for både brugeroplevelsen (UX) og udviklingshastigheden. Det valgte framework skal understøtte en komponentbaseret arkitektur og integrere sømløst med projektets backend-teknologi.

Følgende Tabel 7 sammenligner fire af de mest markedsledende frameworks anno 2026 (31, 32):
React (Next.js), Vue.js, Angular og Svelte:

Tabel 7: Sammenligning af frontend-frameworks baseret på fordele og ulemper i forhold til projektets krav.

Frontend-framework			
Teknologi	Fordele	Ulemper	Valgt?
React (Next.js) (4, 33)	Økosystem: Et af de største fællesskaber og biblioteksudvalg (32, 34). Fleksibilitet: Komponentbaseret arkitektur der er let at genbruge. Integration: Next.js tilbyder indbygget routing og SSR (Server-Side Rendering). TypeScript: Fremragende understøttelse.	Læringskurve: Kan være stejl pga. JSX og hooks-koncepter. Boilerplate & Komplexitet: Selvom Next.js løser routing, kræver React-økosystemet ofte mere boilerplate-kode til state-management (f.eks. Context API/Zustand) sammenlignet med de mere indbyggede reaktive systemer i Svelte eller Vue.	☑
Vue.js (35)	Simplicitet: Meget let at lære og komme i gang med. Performance: Letvægts og hurtig rendering. Struktur: Klar adskillelse af HTML, CSS og JS i filer.	Økosystem: Mindre end Reacts (færre tredjepartsbiblioteker). TypeScript: God support, men historisk set ikke så stærk som Reacts.	☒
Angular (36)	Komplet pakke: "Batteries included" (Routing, Forms, HTTP er indbygget). Struktur: Tvunget arkitektur der sikrer ensartethed. Type-sikkerhed: Bygget op omkring TypeScript fra bunden.	Kompleksitet: Meget tungt og komplekst framework. Fleksibilitet: Mindre fleksibelt end React og Vue. Overhead: For meget boilerplate til et enkeltmandsprojekt.	☒

Svelte (37)	Performance: Ingen Virtual DOM, kompilerer til ren JS. Syntaks: Meget lidt boilerplate-kode. Reaktivitet: Indbygget og meget intuitivt.	Økosystem: Det mindste af de fire, færre jobmuligheder og biblioteker. Modenhed: Mindre battle-tested i enterprise-miljøer end React/Angular.	
-------------	---	---	---

Ud fra ovenstående tabelsammenligninger, er valget faldet på React, implementeret via Next.js.

3.1.1. Valg og begrundelse: modent økosystem og fullstack-kapabilitet

Valget af React (Next.js) er truffet for at maksimere udviklingseffektiviteten og fremtidssikre applikationen. Selvom Vue og Svelte tilbyder en lavere læringskurve, vælges React på grund af det massive økosystem. Som enkeltudvikler er det en afgørende fordel at kunne trække på et bredt udvalg af veltestede komponentbiblioteker (som Shadcn/UI (38) eller Radix UI (39)), der primært er bygget til React (33), hvilket accelererer udviklingen af brugergrænsefladen markant. Desuden har enkeltmandsudvikleren allerede forudgående praktisk erfaring med React/Next.js, hvilket reducerer onboarding-tid og risiko for forsinkelser.

Denne tilgang gør det muligt at samle applikationen i et samlet modulært system med tydeligt opdelt moduler (UI-komponenter, API routes m.m.) fremfor spredte services (separation of concerns). Det minimerer antallet af integrationer, men kræver samtidig en disciplineret adskillelse af UI, logik og datatyper, så de forskellige lag ikke bliver for afhængige af hinanden.

Angular vurderes som for tungt og rigidt til dette projekt, da den tid, der ville blive brugt på frameworkets boilerplate, bedre kan investeres i at udvikle værdiskabende features i React.

Next.js tilføjer desuden indbyggede API Routes (Route Handlers) (9), som gør det muligt at bygge en ægte fullstack monolit. Dette eliminerer behovet for et kompliceret Backend-For-Frontend (BFF) (40) lag og en ekstern backend-tjeneste. I stedet håndteres sikkerhedskritiske opgaver, såsom databasekald og integration med Auth.js (16), direkte og sikkert på serveren via Server Components og Server Actions, uden at udsætte forretningslogik eller databasestrenge for klienten. Derved opnås Next.js' SSR-fordele kombineret med en streng og sikker adskillelse af brugerfladelogik og applikationslogik i én samlet kodebase.

3.2. Responsivt og adaptivt design

Da systemet skal anvendes af en brugergruppe med varierende energiniveauer og fysiske begrænsninger, er det essentielt, at brugergrænsefladen er fuldt responsiv og fungerer optimalt på tværs af mobil, tablet og desktop (jf. UR2 i kravspecifikationen (23)). Valget af CSS-tilgang skal sikre en konsistent visuel stil og en effektiv udviklingsproces.

Følgende Tabel 8 sammenligner fire udbredte tilgange til styling af webapplikationer: Tailwind CSS, Bootstrap, CSS Modules og SASS/SCSS:

Tabel 8: Sammenligning af CSS-tilgange baseret på fordele og ulemper i forhold til projektets krav.

Responsivt og adaptivt design			
Teknologi	Fordele	Ulemper	Valgt?
Tailwind CSS (41)	Udviklingshastighed: Styling sker direkte i HTML/JSX, hvilket eliminerer skift mellem filer. Konsistens: Bygget op omkring et fast design-system (tokens). Performance: Fjerner automatisk ubrugt CSS i build-fasen.	Læringskurve: Kræver kendskab til specifikke klassenavne. Læsbarhed: Lange lister af klasser kan gøre koden uoverskuelig initialt.	☒
Bootstrap (42)	Færdige komponenter: Meget hurtigt at opsætte standard-layouts. Velkendt: Stor udbredelse og dokumentation	Customisering: Svært at fravige "Bootstrap-looket" uden kompleks overskrivning. Vægt: Inkluderer meget ubrugt kode (større bundle size).	☐
CSS Modules	Isolation: Forhindrer klassekonflikter ved at scope CSS til komponenter. Standard: Bruger standard CSS-syntaks.	Hastighed: Kræver manuel navngivning af alle klasser (f.eks. BEM-metodik). Boilerplate: Skaber mange små CSS-filer, der skal vedligeholdes.	☐
SASS / SCSS (43)	Funktionalitet: Understøtter variabler, nesting og mixins. Struktur: Gør det muligt at organisere store mængder CSS logisk.	Vedligeholdelse: Kræver manuel styring af medieforespørgsler (responsive breakpoints). Setup: Kræver en kompileringsproces.	☐

Ud fra ovenstående tabelsammenligninger, er valget faldet på Tailwind CSS som CSS-tilgang.

3.2.1. Valg og begrundelse: UI-framework og designimplementering

Valget af Tailwind CSS som CSS-tilgang er primært begrundet i teknologiens evne til at gøre udviklingsprocessen langt hurtigere gennem sin utility-first tilgang (41). Ved at anvende prædefinerede utility-klasser direkte i komponenterne elimineres behovet for at opfinde og vedligeholde komplekse klassenavne, hvilket reducerer den kognitive belastning for udvikleren og minimerer risikoen for CSS-konflikter.

Tailwind CSS understøtter desuden en mobile-first tilgang på en meget intuitiv måde, hvilket er afgørende for at opfylde kravene til et adaptivt layout. Da Tailwind kun inkluderer den anvendte CSS i det endelige produkt, sikres en optimal performance med små filstørrelser, hvilket forbedrer indlæsningstiden (jf. PR1 i kravspecifikationen (23)).

Sammenlignet med Bootstrap tilbyder Tailwind en langt højere grad af designfrihed, hvilket gør det muligt at skabe et unikt visuelt udtryk til systemet uden at skulle kæmpe mod prædefinerede komponent-styles.

4. Udviklings- og deployment værktøjer

I dette afsnit analyseres de værktøjer og metoder, der anvendes til at understøtte udviklingsprocessen, versionsstyringen og den endelige deployment af systemet. Valget af disse værktøjer har til formål at automatisere manuelle processer og reducere tidsforbruget på administration, hvilket er essentielt i et enkeltmandsprojekt.

4.1. Versionsstyring (Git/GitLab)

Valget af repositorystruktur er afgørende for, hvordan kildekoden organiseres, og hvordan CI/CD-pipelines konfigureres. Der skelnes primært mellem en Mono-repo tilgang (alt i et repository) og en Poly-repo tilgang (opdelt i flere repositories).

Følgende Tabel 9 sammenligner Mono-repo og Poly-repo tilgangen i forhold til projektets krav:

Tabel 9: Sammenligning af repositorystrukturer baseret på fordele og ulemper i forhold til projektets krav.

Versionsstyring (Git / GitLab)			
Teknologi	Fordele	Ulemper	Valgt?
Mono-repo (Et samlet repository)	Type-delning: Nemt at dele kode og typer mellem frontend og backend. Atomic Commits: Muliggør ændringer i både API og klient i samme commit.	Kompleksitet: Kræver avancerede værktøjer (f.eks. TurboRepo eller Nx) for at fungere effektivt. Pipelines: Sværere at konfigurere CI/CD, da man kun vil bygge det, der er ændret.	☒
Poly-repo (Separate repositories)	Isolation: Hver del (Frontend, Backend) har sit eget liv og egne pipelines. Simplicitet: Mindre risiko for merge-konflikter og uoverskuelig Git-historik. Fokus: Gør det lettere at arbejde på én User Story ad gangen.	Type-delning: Sværere at dele TypeScript-interfaces direkte uden en package-manager; kan afhjælpes via en lille intern shared package (npm workspace / privat npm-package). Overblik: Kræver styring af flere separate Git-historikker.	☐

Ud fra ovenstående tabelsammenligninger, er valget faldet på en Mono-repo struktur.

4.1.1. Valg og begrundelse: repositorystruktur og workflow

I direkte forlængelse af beslutningen om at benytte Next.js som fullstack-framework (jf. afsnit 2.1.1), fravælges Poly-repo strukturen til fordel for et Single Repository på GitLab (44).

Da både frontend og backend (API routes) lever i det samme Next.js-projekt, elimineres behovet for at styre flere repositories, og opsætningen af CI/CD-pipelines forenkles drastisk. Den absolut største fordel er dog den indbyggede type-delning. Prisma-genererede typer som Dagsform, Opslag og Bru-ger er øjeblikkeligt tilgængelige for både server-komponenter og klient-komponenter, uden at man skal bygge, udgive og vedligeholde interne npm-pakker. Dette valg er en afgørende forudsætning for at holde projektet agilt og overkommeligt for en enkeltmandsudvikler.

De nævnte typiske ulemper i Tabel 9 ved Mono-repo-strukturer - behovet for avancerede build-værktøjer som TurboRepo eller Nx og kompleks differentieret CI/CD - gælder primært store multi-pakke setups med mange uafhængige applikationer. I Chroniqos tilfælde er repositoryet ét samlet Next.js-projekt, der håndterer bundling, routing og type-delning internt, hvorfor ingen af disse ulem-per gør sig gældende.

4.2. Opgavestyring (Trello/Kanban)

For at sikre fremdrift og overblik over projektet er det nødvendigt med et visuelt værktøj til opgavestyring.

Følgende Tabel 10 sammenligner tre udbredte værktøjer til opgavestyring og projektflow: Trello (Kanban), Jira og GitLab Issues:

Tabel 10: Sammenligning af opgavestyringsværktøjer baseret på fordele og ulemper i forhold til projektets krav.

Opgavestyring (Trello / Kanban)			
Teknologi	Fordele	Ulemper	Valgt?
Trello (45)	Visuelt: Meget intuitivt drag-and-drop interface. Simplicitet: Ingen kompleks konfiguration kræves. Fleksibilitet: Nemt at tilpasse lister til projektets faser.	Integration: Mindre tæt kobling til kildekoden end indbyggede værktøjer.	☒
Jira (46)	Avanceret: Kraftfulde rapporterings- og planlægningsværktøjer. Standard: Udbredt i større organisationer i branchen (47).	Overhead: For tungt og komplekst til et enkeltmandsprojekt.	☐
GitLab Issues (48)	Integration: Direkte kobling mellem issues og commits/merge requests. Alt-i-ét: Samler alt et sted.	Visuelt overblik: Kan føles mindre overskueligt som rent tavleværktøj sammenlignet med Trello.	☐

Ud fra ovenstående tabelsammenligninger, er valget faldet på Trello, som opgavestyringsværktøj.

4.2.1. Valg og begrundelse: opgavestyring og projektflow

Valget af Trello (45) som det primære værktøj til opgavestyring via en Kanban-tavle er truffet for at prioritere et visuelt og lettilgængeligt overblik over projektets status.

Trello gør det muligt hurtigt at flytte opgaver (User Stories) gennem faserne *Backlog*, *To Do*, *In Progress*, *Review*, *Done - Implementering* og *Done - Dokumentation*, hvilket giver øjeblikkelig feedback på projektets fremdrift. For en enkeltmandsudvikler giver Trello den rette balance mellem funktionalitet og hastighed, da der ikke skal bruges unødigt tid på administration, hvilket frigiver ressourcer til selve udviklingsarbejdet.

4.3. Hosting & deployment (cloud-platform)

Valget af hosting-platform har afgørende betydning for systemets tilgængelighed, vedligeholdelse og deploymentshastighed. Da systemet arkitektonisk udvikles som en Next.js fullstack monolit (jf. afsnit 2.1.1), kræves løsninger, der understøtter serverless API-routes, Edge-netværk og en relationel database.

Følgende Tabel 11 sammenligner tre udbredte hosting-strategier: Managed Cloud, Traditionel VPS og On-premise:

Tabel 11: Sammenligning af hosting-strategier baseret på fordele og ulemper i forhold til projektets krav.

Hosting og deployment (cloud-plattform)			
Teknologi	Fordele	Ulemper	Valgt?
Managed Cloud (Vercel / Upstash / Neon) (22, 49, 50)	Automatisering: Fuld CI/CD-integration med Git-Lab. Serverless DB: Neon tilbyder automatisk skalering og dvaletilstand. Drift: Minimeret ansvar for servervedligeholdelse og sikkerhedsopdateringer.	Vendor Lock-in: Afhængighed af specifikke cloud-udbydere systemer. Omkostninger: Kan stige markant ved meget høj trafikvolumen.	☒
Traditionel VPS (Virtual Private Server)	Fuld kontrol: Adgang til alle lag i operativsystemet og server-konfigurationen. Fast pris: Forudsigelige månedlige driftsomkostninger.	Overhead: Kræver manuel opsætning af firewalls, SSL og deployment-scripts. Skalering: Kræver manuel håndtering af ressourceallokering.	☐
On-premise (Egen fysisk server)	Data-suverænitet: Fuldstændig kontrol over fysisk hardware og datalagring. Ingen leje: Ingen løbende udgifter til eksterne hosting-udbydere.	Risiko: Kræver vedligeholdelse af fysisk infrastruktur (strøm, køling, netværk). Setup: Høj kompleksitet og startomkostning.	☐

Ud fra ovenstående tabelsammenligninger, er valget faldet på en Managed Cloud løsning til hosting og deployment af projektet.

4.3.1. Valg og begrundelse: cloud-infrastruktur og serverless hosting

Valget af Managed Cloud som løsning til hosting og deployment af projektet, er begrundet i ønsket om at minimere det administrative ressourcekrav forbundet med serverdrift (NoOps), hvilket frigiver ressourcer til selve systemudviklingen. Der er valgt en infrastruktur baseret på Vercel (22) til hosting af Next.js-applikationen (både frontend og API-routes), Upstash (50) til serverless Redis (21) og Neon.tech (Neon) (49) til den relationelle database.

Anvendelsen af Vercel muliggør en fuldautomatisk CI/CD-pipeline via GitLab, hvorved systemet opdateres øjeblikkeligt ved hver kodeændring. Som en del af samme pipeline bygges og pushes et Docker-image (51) til Docker Hub ved hvert succesfuldt merge til `production`-branchen, tagget med både commit-SHA og `:latest` for præcis versionsgendannelse. Docker anvendes udelukkende som pakkeformat for produktionsimaget - lokal udvikling foregår direkte på værtsmaskinens Node.js-installation uden brug af Docker Desktop. Dette understøtter den iterative udviklingsproces og sikrer et kontinuerligt udviklingsflow.

Valget af Neon (49) som database-host er specifikt truffet pga. deres serverless arkitektur. Neon muliggør skalering til nul, hvilket betyder, at databasen går i dvale ved inaktivitet, hvilket er ressourcebesparende. Samtidig løser Neon komplekse udfordringer med forbindelseshåndtering (connection pooling) i serverless miljøer, hvilket sikrer en stabil integration med Prisma ORM (5) og Next.js' API-routes (9).

Ved at satse på en fuld serverless-arkitektur, sikres et system, der er uafhængigt af en specifik serverinstans. Dette kræver et særligt fokus på, hvordan tilstand (*state*) håndteres. Rate limiting og midlertidig caching styres derfor uden at binde state til serveren via Upstash Serverless Redis.

Samlet set sikrer denne cloud-baserede tilgang en moderne, skalerbar og vedligeholdelsesvenlig infrastruktur.

4.4. UI/UX-designværktøjer

For at sikre en høj brugervenlighed og et sammenhængende visuelt design, er det nødvendigt med en struktureret designfase, før selve implementeringen påbegyndes. Dette sikrer, at layout-beslutninger er gennemtænkte, testede og lever op til UI/UX-standarder.

Følgende Tabel 12 sammenligner tre metoder til design og prototyping: Figma, Adobe XD og Skitsering på papir:

Tabel 12: Sammenligning af designværktøjer baseret på fordele og ulemper i forhold til projektets krav.

UI/UX-designværktøjer			
Teknologi	Fordele	Ulemper	Valgt?
Figma (52)	Prototyping: Muliggør interaktive klikmodeller. Design-system: Stærk understøttelse af genbrugelige komponenter. Web-baseret: Kræver ingen installation og er let at dele.	Online krav: Kræver stabil internetadgang for at fungere optimalt.	☒
Adobe XD (53)	Integration: Fungerer godt med andre Adobe-produkter. Offline: Kan anvendes uden internetforbindelse.	Fremtidssikring: Officiel udvikling er stoppet, hvilket gør det mindre fremtidssikkert.	☐
Skitsering på papir	Hastighed: Meget hurtig metode til indledende idégenerering. Tilgængelighed: Kræver ingen tekniske forkundskaber.	Præcision: Kan ikke anvendes som direkte reference for CSS-værdier eller komplekse interaktioner.	☐

Ud fra ovenstående tabelsammenligninger, er valget faldet på Figma som UI/UX Designværktøj.

4.4.1. Valg og begrundelse: UI/UX-design og designsystem

Valget af Figma (52) som det primære værktøj til udarbejdelse af systemets UI/UX-design er truffet baseret på Figmas evne til let at kunne bygge bro mellem design og implementering. Ved at opbygge et komplet design-system i Figma - inklusiv farvevariabler (tokens), typografi og genbruge- lige komponenter - sikres en visuel rød tråd gennem hele systemet.

Dette letter overgangen til kodning med Tailwind CSS markant, da specifikke værdier for marginer, farver og størrelser kan overføres direkte. Prototyping-funktionaliteten i Figma gør det desuden muligt at validere brugerflows (jf. UR1 i kravspecifikationen (23)), før der investeres tid i kodning, hvilket minimerer risikoen for dyre designændringer sent i forløbet.

4.5. Testværktøjer og kvalitetssikring

For at sikre systemets robusthed og verificere, at den forretningskritiske logik fungerer efter hensigten, er det nødvendigt at implementere et solidt testframework. Da projektet udvikles af en enkeltmandsudvikler, er det afgørende, at testværktøjet har lav konfigurationskompleksitet, stærk TypeScript-understøttelse og integrerer problemfrit med Next.js og GitLab CI.

Følgende Tabel 13 sammenligner tre udbredte testframeworks: Jest, Vitest og Mocha + Chai:

Tabel 13: Sammenligning af testframeworks baseret på fordele og ulemper i forhold til projektets krav.

Testværktøjer og kvalitetssikring			
Teknologi	Fordele	Ulemper	Valgt?
Jest (54)	Økosystem & Integration: Officielt understøttet af Next.js via next/jest, hvilket gør opsætningen triviel. Alt-i-en pakke: Inkluderer test runner, assertions og mocking "out of the box". Modenhed: Enormt community og fuld understøttelse af biblioteker som jest-mock-extended (55) til Prisma.	Hastighed: Lidt langsommere kørsel sammenlignet med moderne, Vite-baserede alternativer som Vitest.	☑
Vitest (56)	Performance: Ekstremt hurtigt grundet Vite-arkitekturen. Moderne API: Kompatibelt med Jest API'et, men bygget til ES-modules.	Kompleksitet: Next.js anvender Turbopack/SWC frem for Vite. Det kræver manuel og til tider skrøbelig konfiguration at tvinge Vitest til at forstå Next.js-specifikke aliaser og server-miljøer.	☒
Mocha + Chai (57, 58)	Fleksibilitet: Modulært opbygget, så man kan vælge præcis de assertion- og mocking-biblioteker (f.eks. Sinon), man ønsker.	Overhead: Gammeldags arkitektur, der kræver omfattende manuel opsætning og vedligeholdelse af flere afhængigheder. Dårligere indbygget TypeScript-support.	☒

Ud fra ovenstående tabelsammenligninger, er valget faldet på Jest som det primære testframework.

4.5.1. Valg og begrundelse: Jest som testframework

Valget af Jest er primært begrundet i teknologiens native integration ("out of the box") med Next.js. Next.js tilbyder en indbygget `next/jest`-konfiguration, der automatisk håndterer TypeScript-transformationen via Next.js Compiler (SWC) (59) samt path aliases (`@/*`), hvilket eliminerer den manuelle opsætningskompleksitet, der er forbundet med alternativer som Vitest i et Next.js-miljø.

En central fordel ved Jest er understøttelsen af `jest-mock-extended`, der muliggør en dyb og fuldt typesikker mocking af Prisma-klienten. Dette er essentielt for at unit-teste Service-laget i komplet isolation, uden at kræve en reel databaseforbindelse under test-kørsel, hvilket optimerer både hastighed og pålidelighed af testsuiten.

For at undgå, at testskrivning udskydes til slutningen af projektet og derved bliver en uoverstigelig byrde, anvendes en inkrementel teststrategi. Tests skrives og eksekveres lokalt i takt med, at de enkelte features færdiggøres. Test-suiten integreres direkte i GitLab CI/CD-pipelinen via JUnit-rapporter (60), der fungerer som en automatiseret kvalitetsport (Quality Gate). Dette sikrer mod regressioner, hver gang ny kode flettes ind i `development`-branchen, og understøtter løbende kodekvaliteten i enkeltmandsprojektet.

5. Samlet konklusion

I dette afsnit opsummeres de teknologiske valg, og der reflekteres over den samlede arkitekturs relevans til at løse projektets problemstilling.

5.1. Overblik over valgt teknologistak

På baggrund af de gennemførte analyser er der sammensat en moderne og robust teknologistak, som danner fundamentet for systemet, hvilket fremgår af Tabel 14 på næste side.

Tabel 14: Samlet overblik over de valgte teknologier i Chroniqo, fordelt på arkitektoniske lag og komponenter. Tabellen opsummerer de endelige teknologivalg på baggrund af de gennemførte analyser og danner fundamentet for systemets implementering. Samtlige valg er motiveret ud fra de fire overordnede nøgleprincipper: Developer Experience & Effektivitet, Skalerbarhed & Performance, Datasikkerhed & Integritet samt Vedligeholdelse & Dokumentation.

Teknologivalg for Chroniqo-projektet	
Kategori / Komponent	Teknologivalg
Fullstack framework	Next.js (App Router) med TypeScript, hostet serverless på Vercel (1, 4, 9, 22)
Styling og responsivitet	Tailwind CSS (41).
Databaseteknologi	PostgreSQL (Relationel) med Prisma ORM, hostet på Neon.tech (5, 10, 49).
Caching og rate limiting	Serverless Redis via Upstash (21, 50).
Autentificering og sikkerhed	Auth.js (NextAuth v5) der abstraherer OAuth 2.0 PKCE-flowet og håndterer stateless JWT-sessioner via sikre HttpOnly-cookies (16, 19, 61, 62).
Realtidskommunikation (chat)	Short-Polling (via SWR) frem for stateful WebSockets, som et bevidst arkitektonisk MVP-valg (27).
AI-integration	Google Gemini API (LLM) implementeret sikkert server-side (24).
Infrastruktur	Docker til container-versionering af produktionsimaget via GitLab CI/CD-pipeline; selve produktionsmiljøet hostes via Managed Cloud på Vercel (22, 51).
Testframework og kvalitetssikring	Jest via next/jest med jest-mock-extended til typesikker Prisma-mocking, eksekveret automatisk i GitLab CI/CD-pipeline. (54, 55)

5.2. Refleksion over teknologivalg (læring og udfordringer)

Det vurderes, at kombinationen af de valgte teknologier er optimalt balanceret i forhold til projektets krav om høj sikkerhed, dataintegritet og udviklingseffektivitet. Ved at samle arkitekturen i en Next.js-monolit og delegerere kompleks sikkerhedslogik til Auth.js, håndteres projektets største risiko (jf. R1: Helbredsudfordringer (2)) via en bevidst minimering af den tekniske kompleksitet.

Den samlede arkitektur håndterer de væsentligste risici identificeret i risikoanalysen, herunder især sikkerhedsmæssige sårbarheder (via PKCE og `HttpOnly`-cookies) og teknisk kompleksitet (via modne og veldokumenterede frameworks). Systemet fremstår dermed teknisk optimeret til både at opfylde MVP-kravene og understøtte fremtidig skalerbarhed og videreudvikling.

Som et afgørende sikkerhedsnet i denne arkitektur fungerer Jest via en GitLab CI-pipeline som garant for, at høj kodekvalitet opretholdes, og at kritiske regressioner fanges tidligt i en ellers agil og hurtig udviklingsproces.

6. Referencer

1. Microsoft Corporation. TypeScript Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.typescriptlang.org/docs/>.
3. PostgreSQL Global Development Group. Appendix M. Glossary [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.postgresql.org/docs/current/glossary.html>.
4. Vercel. Next.js Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://nextjs.org/docs>.
5. Prisma Data. Prisma Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.prisma.io/docs>.
6. OpenJS Foundation. Node.js v25.6.1 Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://nodejs.org/docs/latest/api/>.
7. OpenJS Foundation. Express Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://expressjs.com/>.
8. Python Software Foundation. Python 3.14.3 Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://docs.python.org/3/>.
9. Vercel. Next: Route Handlers [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://nextjs.org/docs/app/getting-started/route-handlers>.
10. PostgreSQL Global Development Group. PostgreSQL 18.2 Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.postgresql.org/docs/18.2/index.html>.
11. Oracle Corporation. MySQL Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://dev.mysql.com/doc/>.
12. DB-Engines. DB-Engines Ranking [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://db-engines.com/en/ranking>.
13. MongoDB. MongoDB Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.mongodb.com/docs/>.
14. Google. Firebase Developer Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://firebase.google.com/docs>.
15. Wilson A, Copplestone P. Supabase Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://supabase.com/docs>.
16. Auth.js contributors. Authjs: Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://authjs.dev/getting-started>.

17. Cox J. Why You Don't Roll Your Own Crypto [Internet]. July 29, 2024. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.vice.com/en/article/why-you-dont-roll-your-own-crypto/>.
18. Clerk. Clerk Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://clerk.com/docs>.
19. Parecki A. Protecting Apps with PKCE [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.oauth.com/oauth2-servers/pkce/>.
20. Parecki A. Authorization Code Exchange [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.oauth.com/oauth2-servers/pkce/authorization-code-exchange/>.
21. Redis Inc. Redis Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://redis.io/docs/latest/>.
22. Vercel. Vercel Documentation [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://vercel.com/docs>.
24. Google. Google AI Studio and Gemini API Documentation [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://ai.google.dev/gemini-api/docs>.
25. OpenAI. OpenAI API Documentation [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://developers.openai.com/api/docs>.
26. Ollama. Ollama: Get up and running with large language models locally [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://ollama.com/>.
27. Svix. Long Polling vs Short Polling [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://www.svix.com/resources/faq/long-polling-vs-short-polling/>.
28. Vercel. Vercel - SWR [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://swr.vercel.app/>.
29. Socket.IO. Socket.io: Documentation [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://socket.io/docs/v4/>.
30. Pusher Ltd. Pusher - Powering realtime experiences for mobile and web [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://pusher.com/>.
31. roadmap.sh. Top 12 Frontend Technologies to Use in 2026: Expert Advice [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://roadmap.sh/frontend/technologies>.
32. Stack Overflow. Web frameworks and technologies [Internet]. 2025. [hentet 23. februar 2026]. Tilgængelig fra: <https://survey.stackoverflow.co/2025/technology#1-web-frameworks-and-technologies>.
33. React Foundation. Quick Start [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://react.dev/learn>.

34. React Foundation. React Community [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://react.dev/community>.
35. You E. Introduction [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://vuejs.org/guide/introduction>.
36. Google. Introduction to the Angular docs [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://v17.angular.io/docs>.
37. Harris R. Svelte Documentation [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://svelte.dev/docs>.
38. shadcn. Introduction [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://ui.shadcn.com/docs>.
39. Grélard B, Moroz V, Hook A, Motta L. Getting started [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://www.radix-ui.com/themes/docs/overview/getting-started>.
40. Calçado P. The Back-end for Front-end Pattern (BFF) [Internet]. 2015. [hentet 23. februar 2026]. Tilgængelig fra: https://philcalcado.com/2015/09/18/the_back_end_for_front_end_pattern_bff.html.
41. Tailwind Labs. Getting started with Tailwind CSS [Internet]. [år ukendt]. [hentet 24. februar 2026]. Tilgængelig fra: <https://tailwindcss.com/docs>.
42. Otto M, Thornton J. Introduction [Internet]. [år ukendt]. [hentet 24. februar 2026]. Tilgængelig fra: <https://getbootstrap.com/docs/4.1/getting-started/introduction/>.
43. Dart Sass. Sass Documentation [Internet]. [år ukendt]. [hentet 24. februar 2026]. Tilgængelig fra: <https://sass-lang.com/documentation/>.
44. GitLab Inc. Find GitLab answers fast [Internet]. [år ukendt]. [hentet 24. februar 2026]. Tilgængelig fra: <https://docs.gitlab.com/>.
45. Atlassian Corporation. Getting started with Trello [Internet]. [år ukendt]. [hentet 24. februar 2026]. Tilgængelig fra: <https://trello.com/guide>.
46. Atlassian Corporation. Jira Documentation [Internet]. [år ukendt]. [hentet 24. februar 2026]. Tilgængelig fra: <https://confluence.atlassian.com/jira>.
47. Atlassian Corporation. 2.5 Enterprise Criteria [Internet]. [år ukendt]. [hentet 24. februar 2026]. Tilgængelig fra: <https://confluence.atlassian.com/enterprise/2-5-enterprise-criteria-461504585.html#:~:text=Availability%20and%20Reliability,Failover%20for%20Jira%20Data%20Center>.
48. GitLab Inc. Issue boards [Internet]. [år ukendt]. [hentet 24. februar 2026]. Tilgængelig fra: https://docs.gitlab.com/user/project/issue_board/.

49. Neon. Neon Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://neon.com/docs/introduction>.
50. Upstash Inc. Upstash - Serverless Data Platform [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://upstash.com/>.
51. Docker. Docker Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://docs.docker.com/>.
52. Figma. Figma Learn [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://help.figma.com/hc/en-us>.
53. Adobe Inc. Adobe XD Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://helpx.adobe.com/xd/user-guide.html>.
54. OpenJS Foundation. Jest Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://jestjs.io/docs/getting-started>.
55. marchaos. Type safe mocking extensions for Jest [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://www.npmjs.com/package/jest-mock-extended>.
56. VoidZero Inc. Vitest Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://vitest.dev/guide/>.
57. OpenJS Foundation. Mocha Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://mochajs.org/>.
58. Chaijs. Chaijs Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://www.chaijs.com/guide/>.
59. Vercel. Next.js Compiler [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://nextjs.org/docs/architecture/nextjs-compiler>.
60. OpenJS Foundation. Junit Reporter: A WebdriverIO reporter that creates Jenkins compatible XML based JUnit reports [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://webdriver.io/docs/junit-reporter/>.
61. Jones MB, Bradley J, Sakimura N. JSON Web Token (JWT) (RFC 7519) [Internet]. 2015. [hentet 25. februar 2026]. Tilgængelig fra: <https://datatracker.ietf.org/doc/html/rfc7519>.
62. The OWASP Foundation. HttpOnly [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://owasp.org/www-community/HttpOnly>.

7. Bilag

2. Villadsen MD. Chroniqo: Risikoanalyse. Bilag 3. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
23. Villadsen MD. Chroniqo: Kravspecifikation. Bilag 1. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.